IEEE FLTA 2026

AuditFL

A Contract-Backed Clearing Layer for Open Federated Learning

Alexandros Balla

Hong Kong University of Science and Technology

Computer Science and Engineering

Spyros Lalis

University of Thessaly

Electrical and Computer Engineering

The participant’s problem

“I contributed an update. What can I check?”

Raw data stay local.
Round decisions still depend on a designated coordinator.

Was my update recorded?

Publication and ticket ownership

What outcome was submitted?

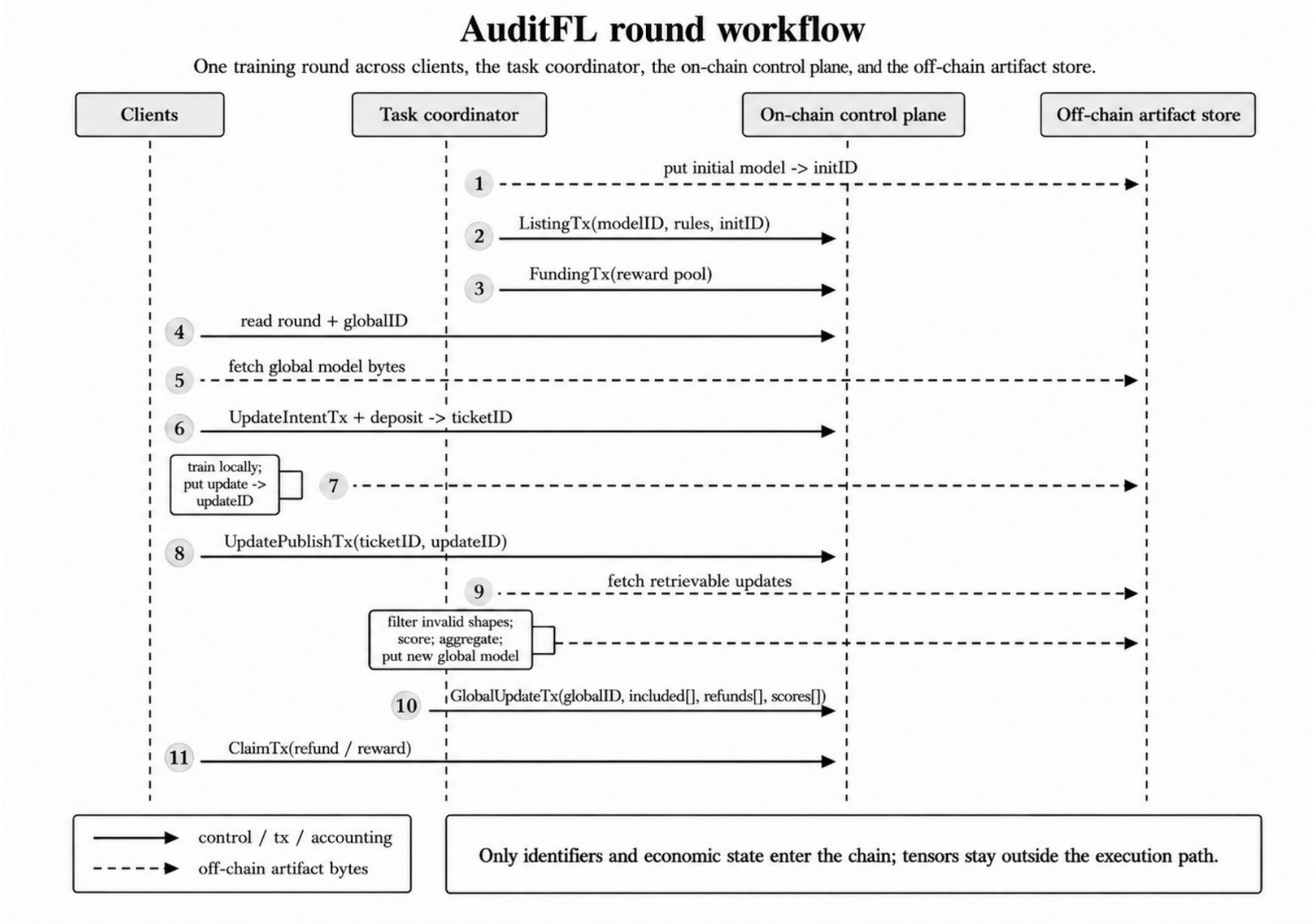
Checkpoint and included updates

Which funds can I claim?

Refunds, rewards and withdrawals

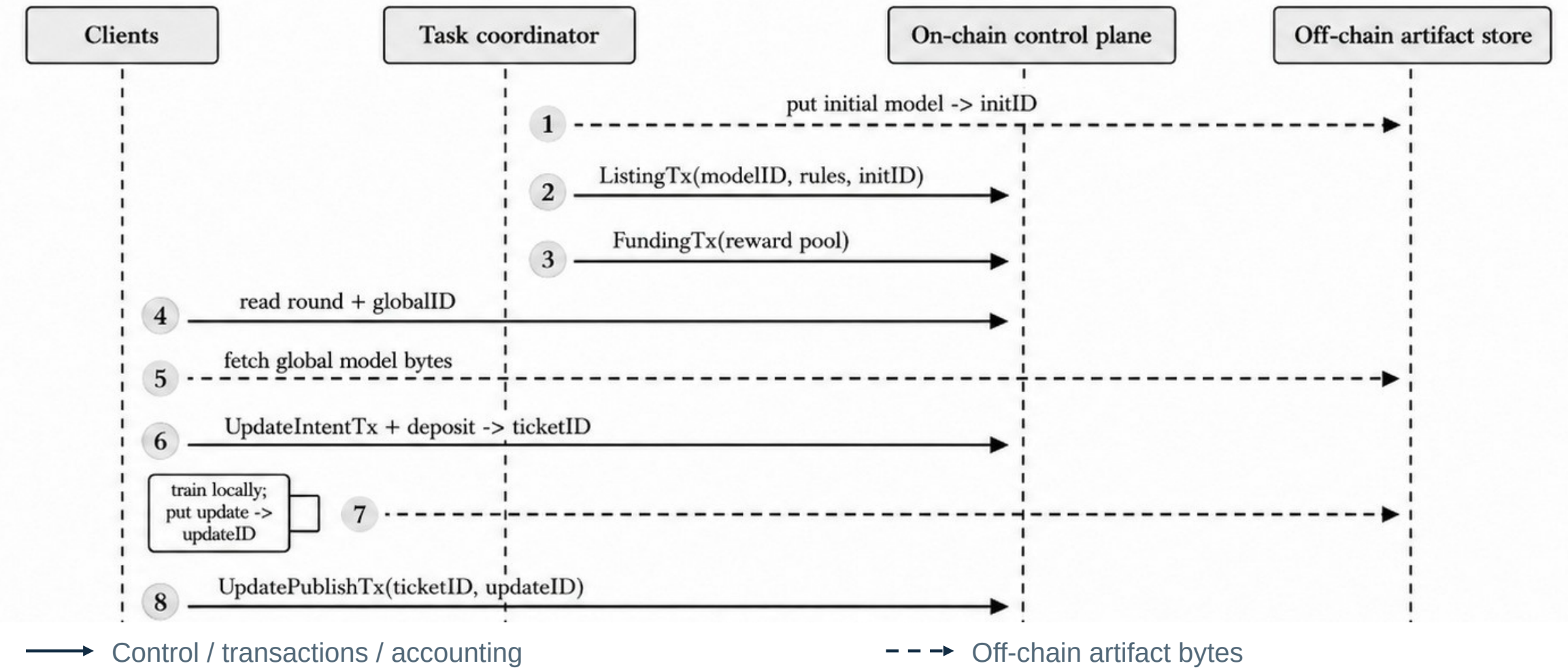
Research question: independently audit participation and settlement

Motivation: final paper, Sec. I



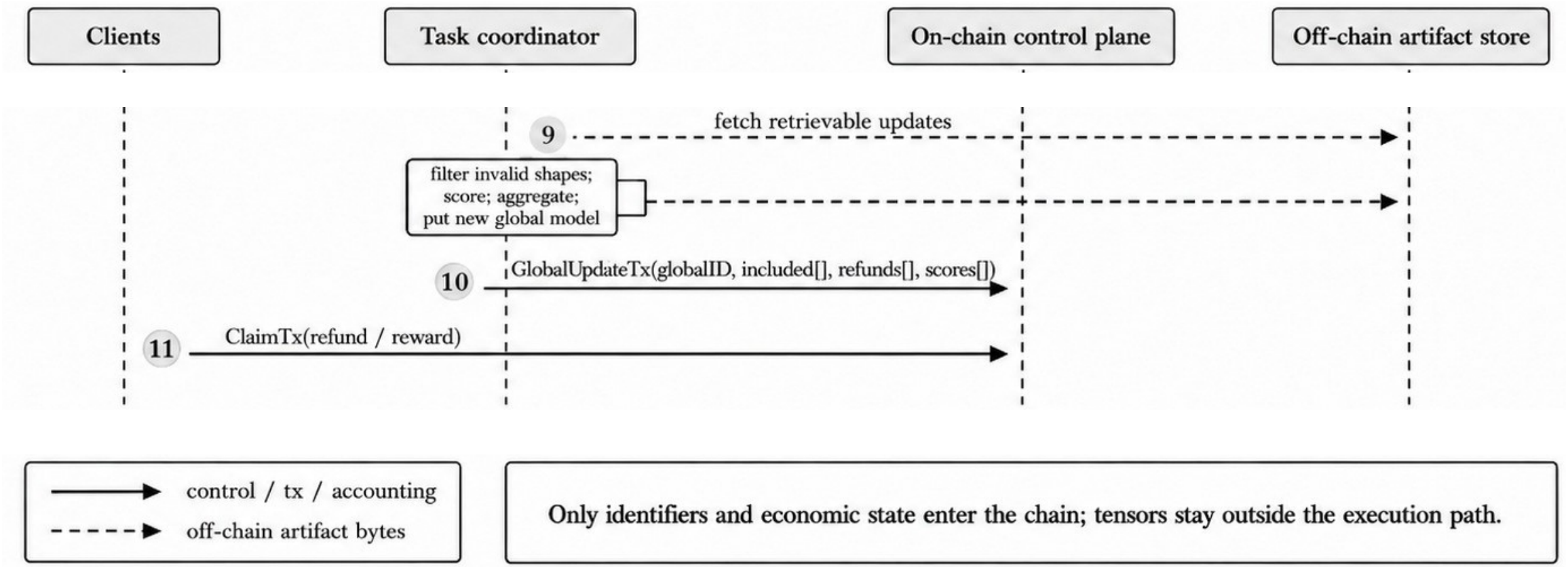
Original paper Fig. 1. Full protocol design; evaluated prototype restrictions on slide 7.

Admission and update publication



Original paper Fig. 1, steps 1–8. Protocol design.

Aggregation and settlement



Protocol figure: scoring and full settlement policy. Prototype: uniform rewards.

The public record allows anyone to

Check the recorded outcome

Ticket ownership and publication
Submitted checkpoint and inclusion
Settlement arithmetic and claims

Replay with retained artifacts

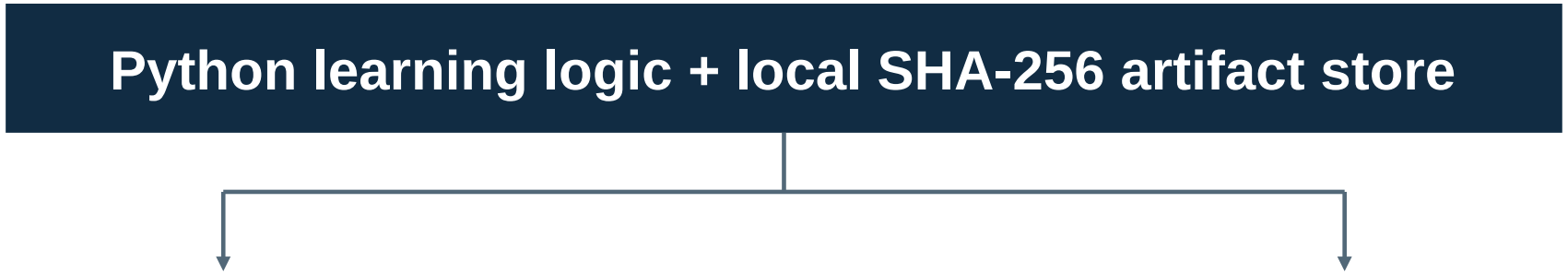
Retrieve the committed artifacts
Rerun the declared procedure
Compare the resulting checkpoint

The contract does not prove honest inclusion, truthful scores or correct aggregation.

Final paper, Secs. III and VI. Replay requires artifacts and a specified deterministic procedure.

The evaluated prototype

MNIST **1–200 clients** **12 rounds** **Non-IID partitions**



Solidity on local Anvil

Transaction counts and gas

In-memory state machine

Controlled failure experiments

Evaluated policy

Uniform rewards per included ticket.
Refunds marked on retrieval, before shape checks.
No deadlines, caps or expiry/forfeiture. Unrefunded deposits stay locked.

Final paper, Sec. VII. Local Anvil measures execution cost; the in-memory backend runs failure stress.

Clearing preserves the clean learning run

MNIST **1–200 clients** **12 rounds**

Matched split, initialization, local training and aggregation

Plain FL		AuditFL
89.21%	=	**89.21%**

Final test accuracy at 200 clients

Clean runs match Plain FL at every evaluated client count.

Final paper, Sec. VIII. All clean comparisons use the same included updates and learning rules.

Counts follow the protocol



200 clients 12 rounds

5,026 transactions

2 + R(2N + 2) + N

2,413 artifacts

1 + R(N + 1)

Measured counts match the model at all 13 tested client counts.

144.9 MiB tensors vs 0.163 MiB identifier strings (not total ledger storage)

Final paper, Sec. VIII-A/B. Counts assume clean full participation and one final claim per client.

Reserve and publish dominate gas

200 clients 12 rounds 5,026 transactions

87.1%

of gas comes from per-client reserve and publish calls



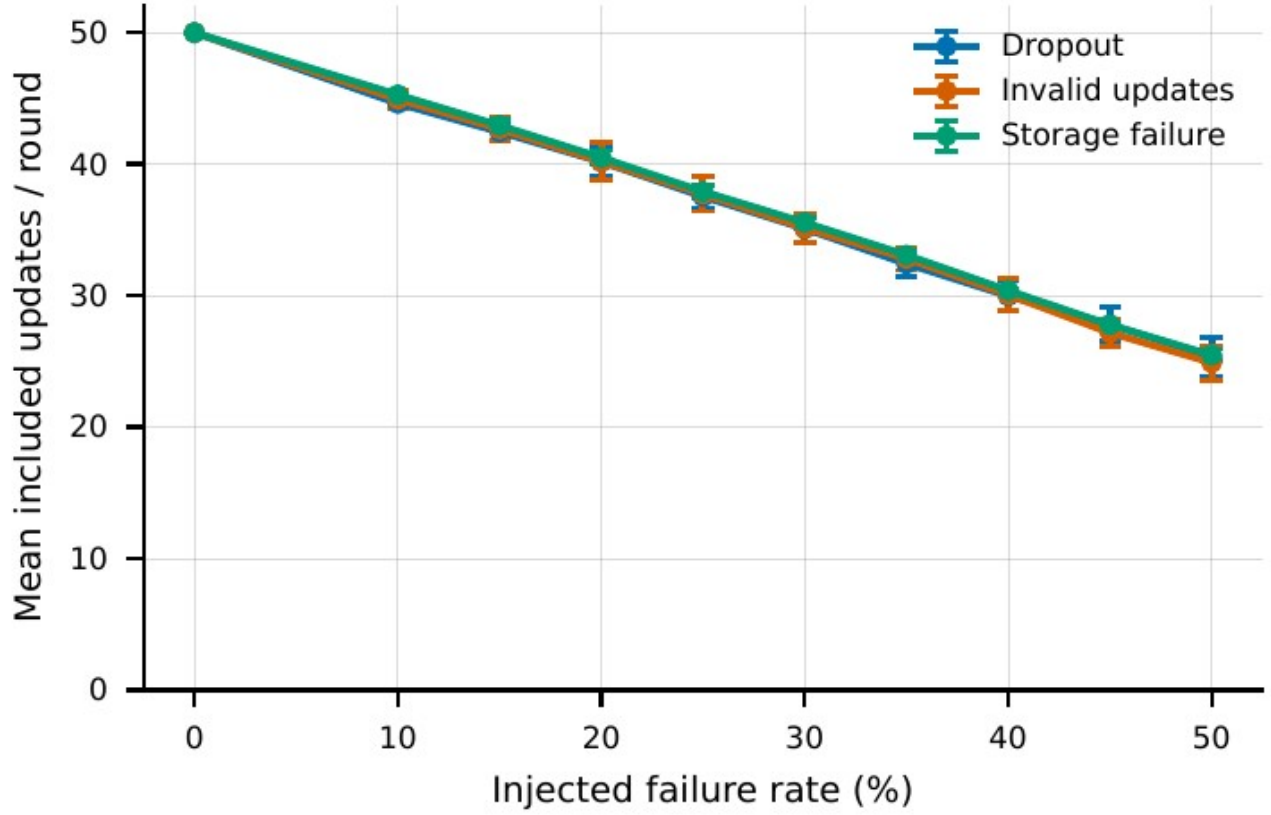
829.7M gas total

Local EVM execution cost
Public-network fees and latency remain unmeasured

Final paper, Sec. VIII-B. Other gas is the residual of rounded reported totals.

Rounds finish despite missing updates

50 clients 12 rounds 0–50% injected failures



All tested rounds finalized

≈25

included updates per round at 50% failures

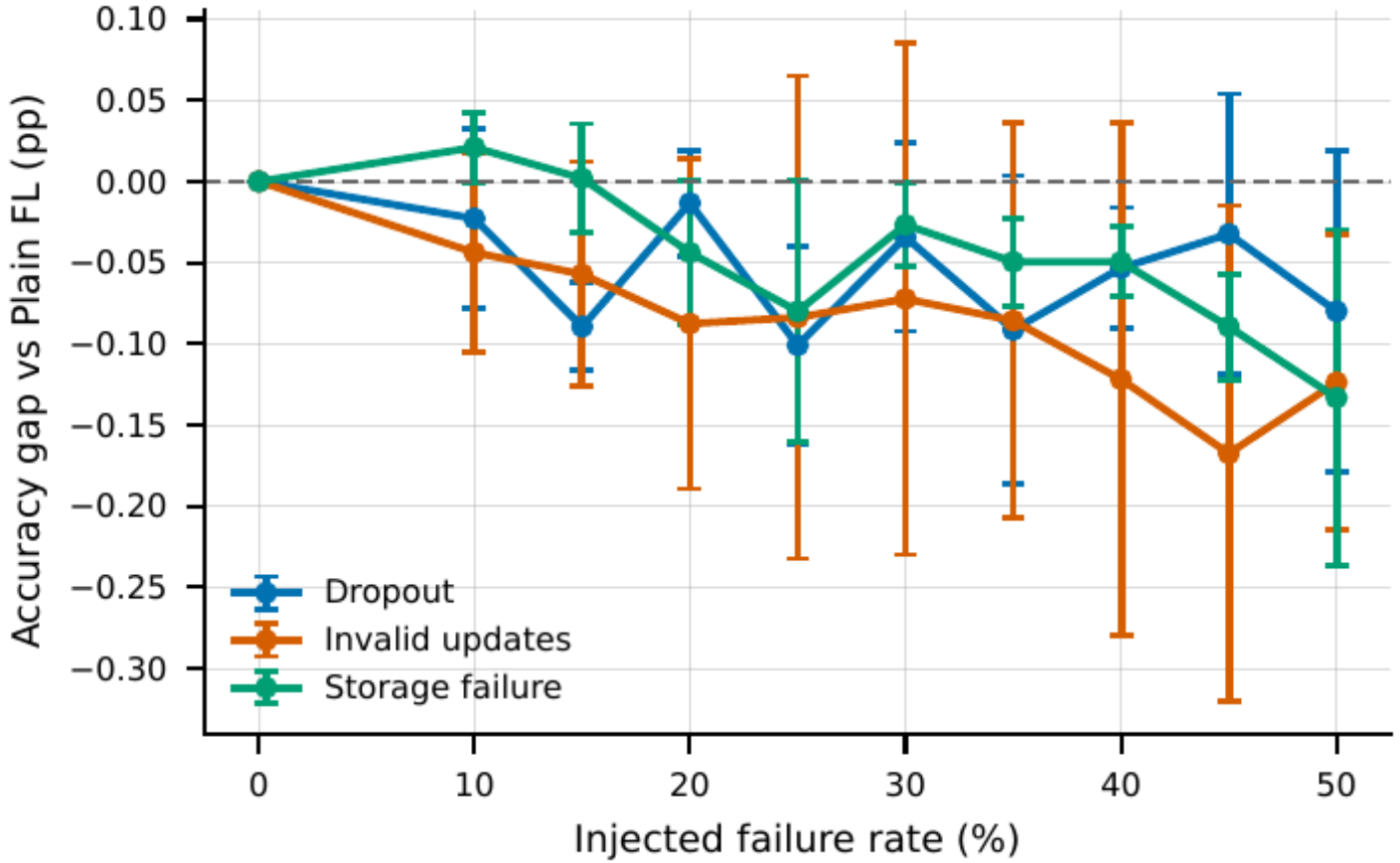
3 fault modes

Dropout, invalid updates and unavailable artifacts

Paper Fig. 5. In-memory stress tests with an operating coordinator. Three seeds.

Accuracy under contribution failures

50 clients 12 rounds 3 seeds



≈0.35 pp

Largest absolute gap reported in the paper

Gap = AuditFL under failure minus Plain FL

Three-seed results are descriptive. They do not establish statistical equivalence.

Paper Fig. 6 and Sec. VIII-C. Accuracy differences are in percentage points. Operating coordinator.

AuditFL: A Contract-Backed Clearing Layer for Open Federated Learning

A.Balla, S.Lalis, IEEE FLTA 2026

“I can check my update commitment, the recorded round outcome, and which funds I can claim.”

Clean learning is preserved. Tested faults still allow round completion. Reservation and publication dominate cost. Batching remains future work.

Connect on LinkedIn

Questions

Paper arxiv.org/abs/2610.41278

Code github.com/katastrofh/AuditFL

aballa@cse.ust.hk

WORKFLOW

One round in the prototype

Setup once: upload checkpoint + list task. Each round: fund the reward pool.

01 Client publishes → 02 Coordinator aggregates → 03 Clearing backend settles

Read round + global ID
Reserve ticket + deposit

↓

Fetch checkpoint bytes
Train locally

↓

Upload update bytes
Publish update ID

Read revealed tickets
from clearing state

↓

Retrieve + mark refunds
Check shapes
Aggregate locally

↓

Upload next checkpoint
Submit finalization

Check current round
and ticket references

↓

Advance round
Credit entitlements

↓

Client submits claim
Withdraw credited funds

Public record: ticket + update ID / finalization / entitlement + claim

Q&A / IMPLEMENTATION

Protocol specification and prototype scope

Mechanism	Protocol specification	Evaluated prototype
Rewards	Score-weighted allocation	Uniform per included ticket
Refunds	Retrievable and basic-valid update	Marked on retrieval, before shape checks
Deadlines and caps	Declared and enforced	Not implemented
Expiry and forfeiture	Policy settles unresolved tickets	Unimplemented; non-refunded deposits stay locked
Storage and ledger	Available storage; confirmed ledger	Local SHA-256 store; Anvil or in-memory backend

Measurements characterize the restricted clearing implementation.

PAPER FIGURE 2 / PROTOCOL DESIGN

Paper protocol: round finalization rule

Round finalization rule. For a fixed task and round r: (1) read all round-r tickets from the k-deep chain view and identify the revealed subset; (2) sort revealed tickets by identifier; (3) fetch referenced updates with bounded retries; (4) add usable revealed tickets to `refunds[]`; (5) add shape-valid and score-positive updates to `included[]` and `scores[]`; (6) aggregate included updates off chain; (7) upload the global checkpoint and submit `GlobalUpdateTx`; (8) the ledger advances r only if referenced tickets are valid, submitted arrays align, and unresolved tickets are settled according to policy.

Original paper Fig. 2. Evaluated prototype restrictions: slide 15 / Q&A 2.

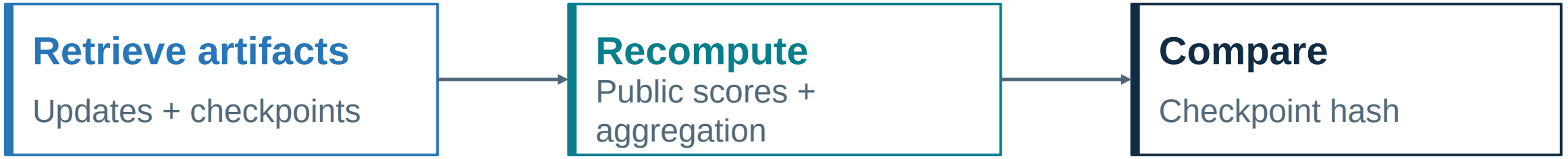
Q&A / TRUST BOUNDARIES

Audit the record, then replay the learning

1 Read the confirmed record

Ticket ownership → publication → submitted inclusion → entitlement → claim

2 Add artifacts and a fully specified procedure



Honest inclusion is not enforced. Coordinator failure can halt finalization.

Q&A / EVALUATION

Experimental setup and fault boundaries

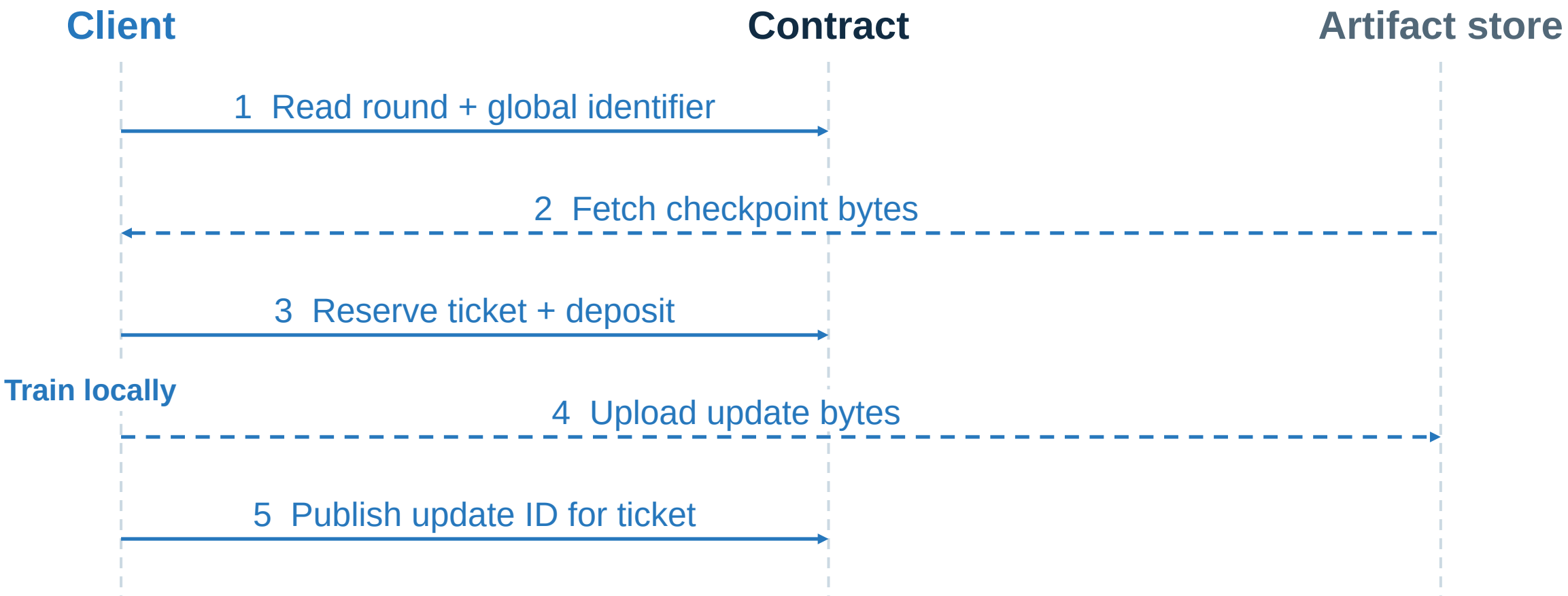
Condition	Configuration
Client counts	1, 2, 5, 10, 15, 20, 30, 40, 50, 75, 100, 150, 200
Data / partition	MNIST 42k train / 10.5k validation / 17.5k test; Dirichlet $\alpha = 0.4$
Model / optimizer	7,850 float64 parameters; 2 full-batch steps; lr 0.35; L2 = 10^{-4}
Aggregation	Sample-count weighted; matched to Plain FL
Fault matrix	50 clients × 12 rounds × 3 seeds; failure rates 0–50%
Fault modes	Dropout, invalid updates, unavailable artifacts

Excluded: coordinator failure, malicious exclusion and poisoning.

Three-seed results are descriptive; they do not establish statistical equivalence.

Client publication sequence

A ticket links one participant, one round and one update identifier



The public record binds the update identifier to a reserved ticket.

Redrawn from final paper Fig. 1, steps 4–8. Setup: initial checkpoint, listing and funded pool.

Contract finalization checks

The coordinator proposes a result. The contract settles the recorded outcome.

Coordinator

Read confirmed, revealed tickets

Fetch updates with bounded retries
Check, score and aggregate locally

Upload the next checkpoint

Finalization

checkpoint ID + included tickets
refunds + scores

Contract

Check round, ticket references and aligned arrays

Apply settlement rules
Advance the round

Credit claimable entitlements

Clients withdraw later through a claim transaction.

Redrawn from final paper Figs. 1–2. Full protocol uses supplied scores for rewards.